Hackathon Platform — Deliverable 1

Team: Group 5
Repo link: https://github.com/KlosRadieschen/Hackathon-Student-Competition-Platform

1. Project scope

The platform runs a university hackathon from start to finish in one place. Organizers publish challenges and deadlines. Students form teams, register for a challenge and submit a project before the deadline. Judges score submissions against a rubric, and organizers publish the results. Competition specific rules such as deadlines, minimum and maximum team size, available challenges, and evaluation criteria are configured by the organizer rather than hard coded into the platform.

2. Stakeholders & needs

Three primary roles use the system directly, the university and IT support influence it.

Stakeholder	Category	RE role	Key needs / concerns
Participant	Primary	Requirement provider,validator	Find a team; see instructions and deadlines; upload and submit work
Organizer	Primary	Requirement provider, decision-maker,validator	Configure challenges, rules and deadlines; monitor registrations and submissions; publish results
Judge	Primary	Requirement provider,validator	See assigned submissions; score with a clear rubric; declare conflicts of interest
University	Secondary	Decision-maker	Fair, reputable event; low running cost; reusable for future editions
IT support / platform admin	Hidden	Maintainer/operator	Manage accounts and roles; stay up under the deadline traffic spike; clear logs and backups

3. User stories

Participant

1. As a participant, I want to browse and search challenges, so that I can pick one that matches my skills.
2. As a participant, I want to create a team or join one through an invite code, so that I don't have to coordinate over email.
3. As a participant, I want to see important competition deadlines so that I can complete required actions on time .
4. As a participant, I want to see my team's score and judge feedback after results are published, so that I can learn from the event.
5. As a participant, I want to be able to register the team for a challenge, so that our team is officially competing.
6. As a participant, I want to be able to upload and update my team's project submission before the deadline so that judges receive the correct and most recent version of our work .

Organizer

1. As an organizer, I want to create challenges with a description, rules and team size limits, so that participants know what is expected.
2. As an organizer, I want to set and, if needed, extend deadlines, so that I can react to problems during the event.
3. As an organizer , I want to define minimum and maximum team sizes so that teams follow the competition rules.
4. As an organizer, I want to assign judges to project submissions so that every project can be evaluated.
5. As an organizer, I want to publish the ranking with one action, so that results reach everyone at the same time.

Judge

1. As a judge, I want to see only the submissions assigned to me, so that I can focus on the projects I need to evaluate .
2. As a judge, I want to score each submission using the organizer defined evaluation criteria so that evaluations are consistent and comparable .
3. As a judge , I want to provide written feedback so that the participants can understand the reasoning behind my evaluation.
4. As a judge, I want to declare a conflict of interest, so that I am not asked to grade a team I know.

Platform admin (hidden)

1. As a platform admin, I want to change user roles and reset accounts, so that I can fix access problems quickly.
2. As a platform admin, I want clear error logs for failed uploads, so that I can help teams near the deadline without involving developers.
3. As a platform admin, I want automated backups of submissions, so that no team loses work.

4. Requirements

Functional

ID	Requirement	Traces to
FR1	Users register and log in securely .	All roles
FR2	The system provides role-based access for participant, organizer, judge, admin	All roles
FR3	Organizers can create, edit and search challenges (title, description, rules, team size min/max)	Organizer 1,Organizer 3
FR4	Organizers can set registration and submission deadlines per challenge, and extend them	Organizer 2
FR5	Participants can create a team, invite members via code/link, and leave a team	Participant 2
FR6	Teams can register for a challenge	Participant 5
FR7	The team can upload or update a submission until the deadline; after it, the submission is locked	Participant 6
FR8	Organizers assign judges to project submissions; judges can declare a conflict of interest	Organizer 4, Judge 4
FR9	Judges score submissions with a configurable rubric and add written feedback	Judge 2, Judge 3
FR10	The system aggregates scores into a ranking; the organizer publishes it	Organizer 5, Participant 4

Non-functional

ID	Requirement
NFR1	**Security:** Passwords are securely stored ; judges can only access assigned submissions; uploaded files scanned and size-limited.

ID	Requirement
NFR2	**Performance:** The system should remain responsive under the expected peak submission workload
NFR3	**Reliability:** Every successful submission gets a timestamped receipt; daily backups; Logging
NFR4	**Usability:** A new participant can join a team and register in under 5 minutes; the platform should work on mobile devices.
NFR5	**Maintainability:** The system should use a modular codebase with appropriate technical documentation and automated testing

5. Discussion: conflicts and assumptions

The biggest tension is between flexibility for participants and fairness across teams.

Potential conflicts

- **Participants vs fairness:** teams want to edit until the last second and want extensions when uploads fail; other teams see extensions as unfair. Possible rule: a short grace period only for technical failures confirmed by logs.
- **Transparency vs judge privacy:** participants want to know who scored them and why; judges may want anonymity to score honestly. Option: show feedback without judge names.
- **Organizer flexibility vs rule enforcement:** organizers want exceptions (a 6-person team), but the system enforces team size. Option: an organizer override with a logged reason.

Open questions

- Can external people join (other universities, industry judges)?
- How is the final score computed: judge average, rubric weights, normalization between strict and lenient judges?
- Are late submissions ever allowed? Is there a grace period?
- Are results public or visible only to participants?

Assumptions

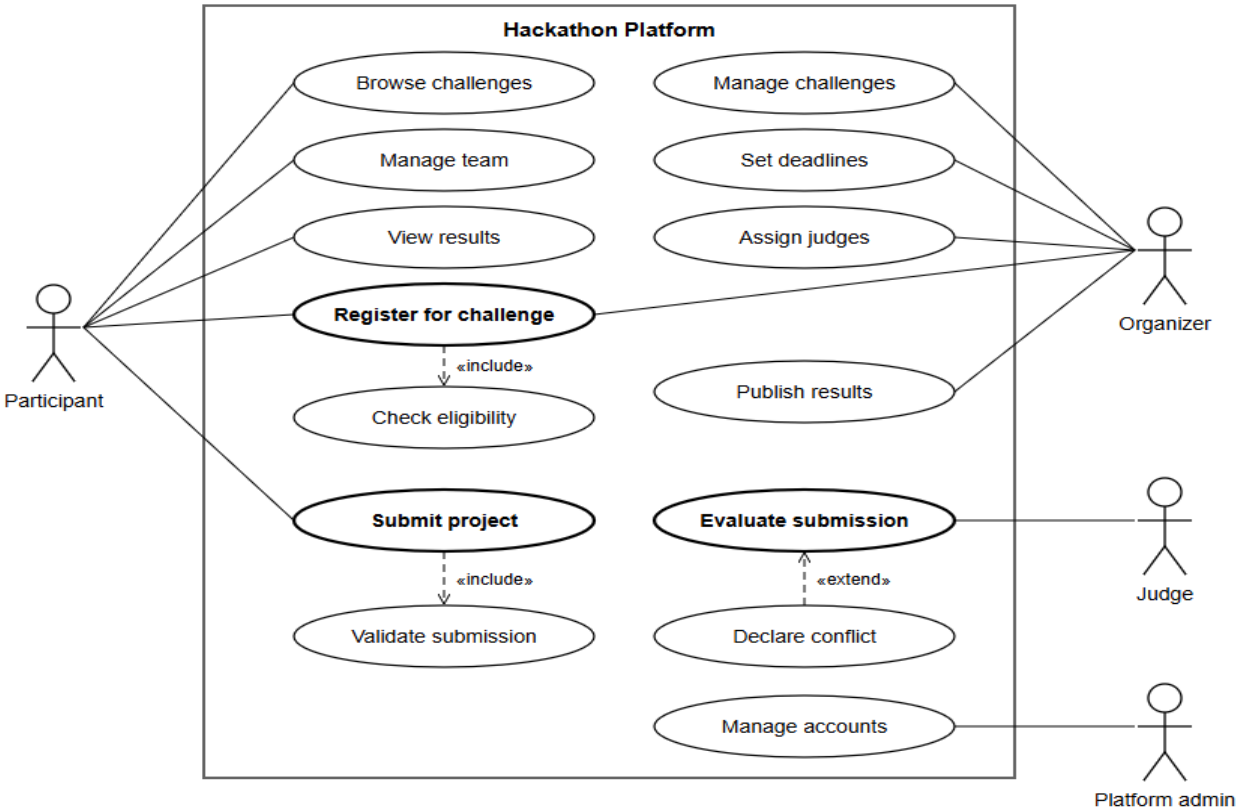
- One challenge per team.
- One team per participant.

6. User story map

The MVP is the smallest set that runs one complete hackathon end to end; Release 1 improves the experience but the event works without it.

Row	Join the event	Form a team	Register for a challenge	Submit a project	Judge & publish
Tasks	Log in · Browse challenges	Create team · Invite members	Pick challenge · Check eligibility	Upload files · Confirm receipt	Score · Rank · Publish
MVP	Email/password login; challenge list with search	Create team; join via invite code	Register button with team-size check	Upload link + files; lock at deadline; receipt	Rubric scoring; automatic ranking; publish button
Release 1		Team-finder board ("looking for teammates")	Waitlist when a challenge is full	Edit until deadline with version history; check repo is reachable	Conflict-of-interest declaration; written feedback;

7. Use case diagram



Main use cases in bold: UC1 Submit project, UC2 Register for challenge, UC3 Evaluate submission

8. Use case descriptions

Three main use cases are described in full; Publish results and Manage team follow the same pattern.

UC1 — Submit project

Field	Description
Actors	Participant (Primary)
Preconditions	Participant is authenticated; Team is registered for a challenge; submission deadline has not passed
Main success scenario	1. A team member opens the team's challenge page and selects "Submit". 2. They enter the repository link, upload files and add a short description. 3. The System **includes** "Validate submission" (deadline, file type/size, required fields). 4. The System stores the submission with a timestamp.
Extensions	3a. Deadline passed: submission rejected, a member sees an error with the deadline time. 3b. File too large or wrong type: system shows which file failed, other data is kept. 4a. Submission already exists: the new version replaces it, the previous one is kept in history.
Postconditions	Exactly one active submission exists for the team, or the team has been told why it failed

UC2 — Register for challenge

Field	Description
Actors	Participant (Primary); Organizer (secondary)
Preconditions	Participant is authenticated; Team exists; registration is open
Main success scenario	1. The team browses challenges and selects one. 2. A member clicks "Register team". 3. System **includes** "Check eligibility" (team size within limits, team not registered elsewhere, challenge not full). 4. System confirms the registration. 5. Team appears in the organizer's registration list.
Extensions	3a. Team too small or too large: registration refused, required size shown. 3b. Challenge full: team offered the waitlist (Release 1).

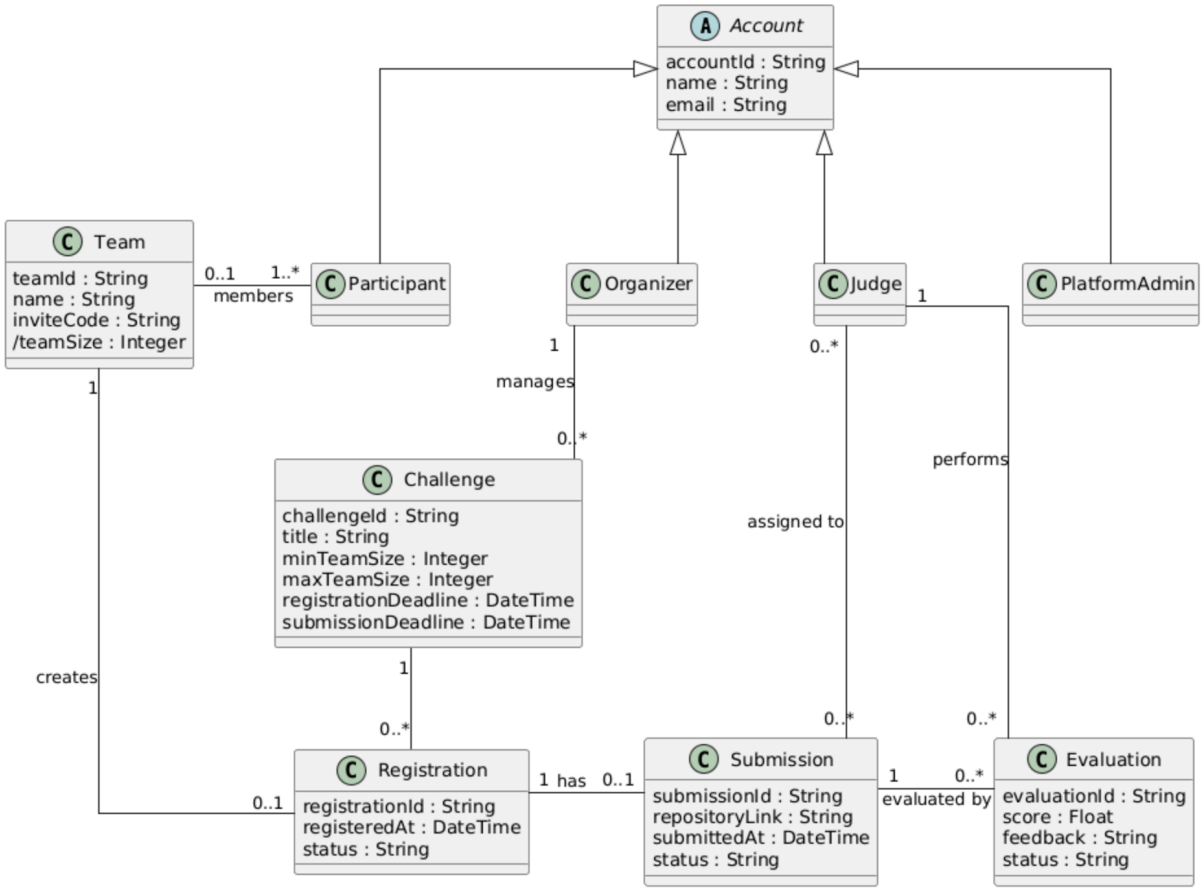
Field	Description
Postconditions	Team is registered, waitlisted, or notified of the reason for failure

UC3 — Evaluate submission

Field	Description
Actors	Judge (primary)
Preconditions	Judge is authenticated and assigned to the project submission; submission deadline has passed
Main success scenario	1. The judge opens the list of assigned submissions. 2. The judge selects a submission and views its files and repository link. 3. The system displays the evaluation rubric 4. The judge scores the submission and writes feedback. 5. The system validates the evaluation and stores it.
Extensions	2a. Judge knows the team: **extend** "Declare conflict"; the submission leaves this judge's list and the organizer is notified to reassign it. Incomplete evaluation :The system informs the judge that the required score or feedback is missing and does not save the document
Postconditions	Evaluation stored, or the submission is flagged for reassignment

9. UML Class Diagram

The following UML class diagram represents the main domain concepts of the Hackathon Platform, their attributes, relationships, generalizations, and multiplicities .



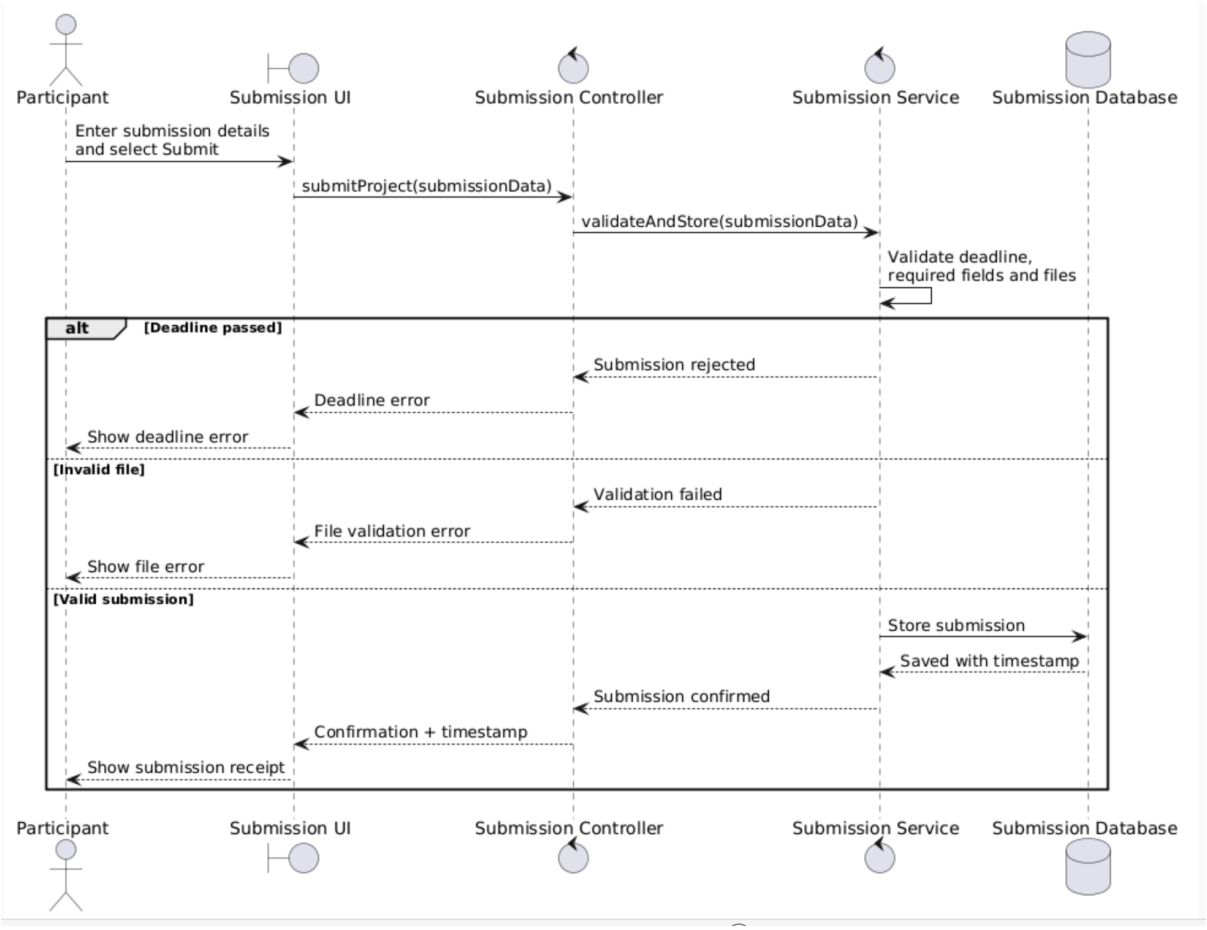
UML class diagram

10. UML Sequence Diagram

The following sequence diagram shows how the main actors and system components interact over time for the main use cases .

10.1 UC1 Submit Project

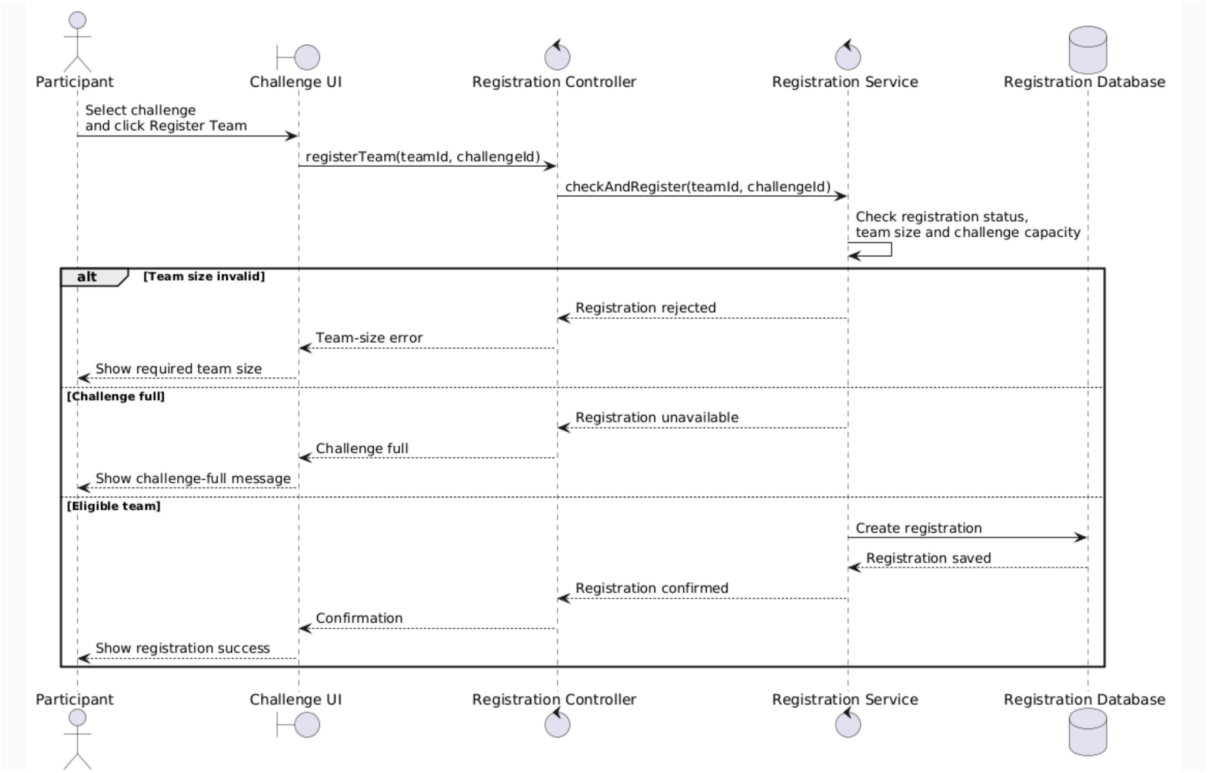
This sequence diagram shows how a participant submits a project . The system validates the submission deadline , required fields , and upload files . if the submission is valid it is stored and the participants receives a confirmation with a timestamp. If the deadline has passed or a file is invalid the participant review an error message



UML sequence diagram submit project

10.2 UC2 Register for Challenge

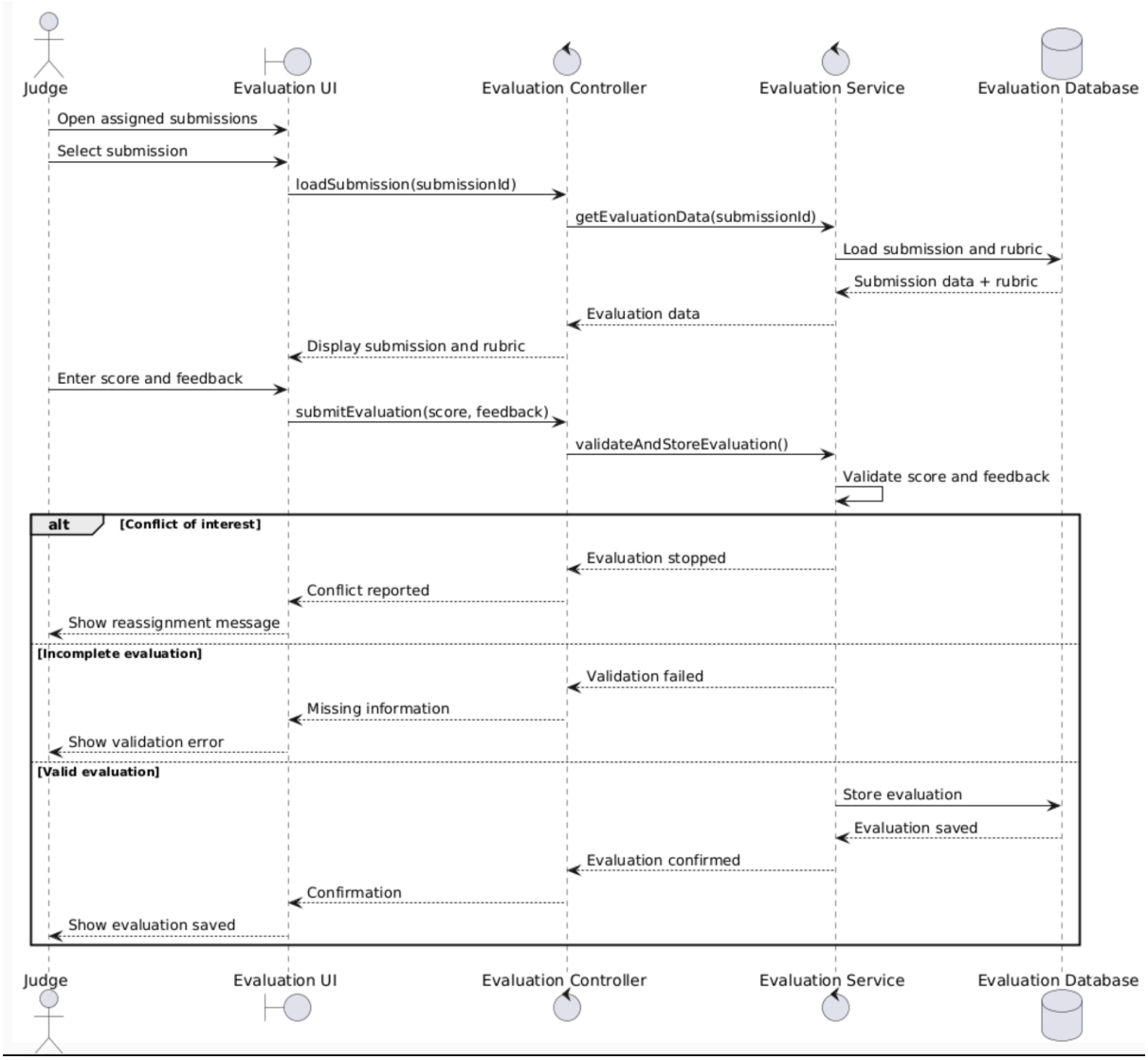
This sequence diagram shows how a participant registers a team for a challenge.The system checks the team's eligibility , including team size rules, existing registration, and challenge capacity. If the team is eligible , the registration is stored and confirmed.Otherwise, the participant is informed why registration failed



UML sequence diagram register for challenge

10.3 UC3 Evaluate Submission

This sequence diagram shows how a judge evaluates an assigned submission .The system loads the submission and evaluation rubric , the judge enters a score and written feedback , and the system validates and stores the evaluation . if a conflict of interest is reported or the evaluation is incomplete , the evaluation is not stored.



UML sequence diagram evaluate submission